

RetailHub E-Commerce & FraudGuard Telemetry Engine

Master Technical Specification & System Architecture Documentation

RetailHub E-Commerce & FraudGuard Telemetry Engine

Master Architectural Specification & System Documentation Book

IMPORTANT:

System Scope: This document serves as the authoritative, end-to-end technical reference and operational manual for the **RetailHub E-Commerce & FraudGuard Real-Time Telemetry Engine**. It covers all system components, data flow pipelines, Kafka event streaming topologies, database schemas, security state machines, UI application pages, API specifications, and Docker containerization blueprints.

1. Executive Summary & Platform Overview

RetailHub is a Next-Generation, Enterprise-Grade E-Commerce Platform integrated with **FraudGuard**, a real-time event-driven telemetry and streaming fraud detection engine.

Key Operational Capabilities

- **Omnichannel E-Commerce Portal:** Full customer shopping journey including catalog discovery, cart management, single-click checkout, order tracking, address management, automated invoice generation, and customer reviews.
 - **FraudGuard Real-Time Security Operations:** Asynchronous telemetry streaming using Apache Kafka to capture user events, analyze behavioral risk indicators, enforce policy state machines, and dynamically safeguard accounts against fraud.
 - **Dynamic Security Workflows:** Multi-tier security enforcement ranging from automated OTP step-up authentication and temporary 24-hour restrictions to administrative manual reviews and permanent account bans.
 - **Resilient Dual-Storage Architecture:** Primary persistence powered by MySQL relational database with an automatic zero-latency In-Memory Fallback Repository to maintain 99.999% availability during database maintenance or outage.
-

2. Platform Technology Stack & Architecture

Core Architecture Components

Layer	Component	Technology / Stack	Purpose
Frontend UI	Web Application	React 18, Vite, TypeScript, Tailwind CSS, Lucide-React, Zustand	Interactive e-commerce portal and FraudGuard Security Ops dashboard.
Backend API	Application Server	Node.js (v20), Express.js, TypeScript, dotenv, CORS, bcryptjs	Handles RESTful API requests, session management, catalog ops, and security rules.
Streaming	Message Broker	Apache Kafka (v2.0 / KafkaJS)	Asynchronous telemetry pipeline for decoupled event streaming and fraud scoring.
Storage	Relational Database	MySQL 8.0 (mysql2/promise)	Relational persistence for customers, orders, incidents, restrictions, and audit logs.
Fallback	Memory Cache	Node.js In-Memory Map Storage	High-speed zero-downtime memory repository fallback if MySQL connection drops.
Packaging	Containerization	Docker, Multi-Stage Dockerfile, Docker Compose	Unified deployment packaging for single-container or microservices deployment.

3. Event-Driven Telemetry & Kafka Streaming Infrastructure

3.1 Event Topics & Mapping Strategy

All application interaction events (clicks, searches, logins, checkouts, password changes, refund requests) are recorded asynchronously and routed to dedicated Kafka topics using `kafkaTopicMapper.ts`.

Domain Category	User Event Type	Kafka Topic Target	Partition Key Strategy
User & Auth	login, login_failed, logout, password_changed, email_changed, phone_changed, device_registered	retail_user_events	customer_id / session_id
Discovery	page_view, category_view, search, product_view, wishlist_add, wishlist_remove	retail_discovery_events	customer_id
Cart	cart_item_added, cart_item_removed, cart_update	retail_cart_events	customer_id

Checkout	checkout_started, address_selected, payment_method_selected, checkout_abandoned	retail_checkout_events	customer_id
Payment	payment_initiated, payment_success, payment_failed, payment_retry	retail_payment_events	customer_id
Order	order_created, order_confirmed, order_cancelled, order_status_updated	retail_order_events	customer_id
Fulfillment	inventory_reserved, shipment_created, order_packed, order_shipped	retail_fulfillment_events	customer_id
Delivery	in_transit, out_for_delivery, delivered, delivery_failed	retail_delivery_events	customer_id
Returns & Refunds	return_requested, return_approved, return_picked_up, refund_initiated, refund_success	retail_return_events	customer_id
Reviews & Ratings	review_added, rating_given	retail_review_events	customer_id
Coupons	coupon_applied, coupon_rejected	retail_coupon_events	customer_id
System	invoice_generated, notification_created	retail_system_events	customer_id
Admin	admin_login, admin_login_failed, admin_dashboard_viewed	retail_admin_events	admin_id
Unrecognized	Unmapped or corrupted events	retail_dead_letter	system

3.2 Kafka Design Rationale: Why Kafka?

NOTE:

Why Apache Kafka over standard HTTP REST or classic queues (RabbitMQ/SQS)?

1. Strict Ordering per Customer: By partitioning topics using **customer_id** as the message key, Kafka guarantees that all sequential events for a single customer arrive in exact chronological order at consumer workers.

2. **Decoupled Telemetry Pipeline:** API responses to e-commerce customers remain under 15ms because telemetry events are emitted asynchronously (**fire-and-forget**) without blocking API execution.

3. **Replayability & Auditability:** Kafka commit logs allow replaying historical customer events to re-train machine learning fraud models or run retrospective security evaluations.

4. **Dead Letter Queue (DLQ):** Corrupted or unmapped event schemas route to **retail_dead_letter**, preventing event drop or pipeline blockages.

4. FraudGuard AI & Real-Time Risk Engine

FraudGuard inspects incoming event telemetry and evaluates complex fraud vectors using a deterministic risk-scoring engine.

4.1 Fraud Categories & Indicators

4.2 Security Status Priority Hierarchy

When multiple fraud events affect a customer, FraudGuard strictly enforces a non-downgrading priority state machine:

\$\$ \text{NORMAL (1)} < \text{STEP_UP_REQUIRED (2)} < \text{PROTECTED (3)} < \text{RESTRICTED (4)} < \text{DEACTIVATED (5)} < \text{BANNED (6)} \$\$

IMPORTANT:

Priority Hierarchy Rule: A security decision action can only **upgrade** or maintain a customer's security status. For example, if a customer is already in **RESTRICTED** status (Priority 4), a subsequent minor event triggering **STEP_UP_REQUIRED** (Priority 2) will **not** downgrade the customer from restriction.

5. Security Workflow & Customer Account Lifecycle

Account Lifecycle Rules & Operational Procedures

1. OTP Step-Up Verification:

- Triggered when risk score is between 30 and 49 or when accessing sensitive endpoints (password change, order placement from new device).
- Generates a 6-digit OTP challenge expiring in 300 seconds (5 minutes).
- Maximum allowed attempts: **3**. Exceeding 3 failed attempts immediately transitions the account to **RESTRICTED** status with a 24-hour lock.

2. Temporary Restriction (24-Hour Lock):

- Triggered automatically by high-risk scores (70–84), multiple failed logins, or failed OTP attempts.
- Restricts checkout, cart addition, profile updates, and payment methods.
- **Expiration & Release Procedure:** When a restriction expires or is released by an admin, the customer **must** pass OTP verification on their next login to regain **NORMAL** active status.

3. Admin Review & Escalation:

- High-severity incidents (score 85+) automatically route to the **Admin Review Queue**.
- Authenticated administrators can perform four actions:
- **Extend Restriction (24h):** Adds temporary restriction time.
- **Deactivate Account:** Suspends account operations.
- **Ban Account:** Permanently blocks customer credentials.
- **Clear Review:** Clears review flag and restores access.

4. Permanent Ban & Unban Revocation:

- Permanently banned customer accounts are logged in **banned_accounts**.
- Administrators can execute **Revoke Ban & Activate** via the Admin Portal, restoring status to **ACTIVE** and security status to **NORMAL**.

6. Complete Database Schema Reference

The platform utilizes 12 primary MySQL tables with full support for relational constraints, foreign keys, and indexes.

6.1 Database Schema Diagram

6.2 Primary Tables & Schema Definitions

1. customers Table

Field	Type	Attributes	Description
customer_id	VARCHAR(64)	PRIMARY KEY	Unique customer identifier (e.g. CUST-1001).
first_name	VARCHAR(64)	NOT NULL	Customer first name.
last_name	VARCHAR(64)	NOT NULL	Customer last name.
email	VARCHAR(128)	UNIQUE, NOT NULL	Login email address.
phone	VARCHAR(32)	NOT NULL	Masked or raw telephone number.
password_hash	VARCHAR(255)	NOT NULL	Bcrypt encrypted password hash.
account_status	VARCHAR(32)	DEFAULT 'ACTIVE'	Status: ACTIVE, SUSPENDED, BANNED.
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Account creation timestamp.

2. fraud_incidents Table

Field	Type	Attributes	Description
incident_id	VARCHAR(64)	PRIMARY KEY	Incident identifier (e.g. INC-20260812-9901).
customer_id	VARCHAR(64)	NOT NULL	Associated customer ID.
fraud_type	VARCHAR(64)	NOT NULL	Type: ACCOUNT_TAKEOVER , CARD_TESTING , etc.
severity	VARCHAR(32)	NOT NULL	Severity level: LOW , MEDIUM , HIGH , CRITICAL .
risk_score	INT	NOT NULL	Risk score (0 to 100).
risk_level	VARCHAR(32)	NOT NULL	Risk band: LOW , MEDIUM , HIGH , VERY_HIGH , CRITICAL .
action	VARCHAR(64)	NOT NULL	Action: MONITOR , STEP_UP , RESTRICT , ADMIN_REVIEW .
requires_admin_review	BOOLEAN	DEFAULT FALSE	Flag for human administrator review queue.
ip_address	VARCHAR(45)	NULLABLE	Client IP address at event detection.
timestamp	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Incident detection timestamp.

3. fraud_restrictions Table

Field	Type	Attributes	Description
restriction_id	VARCHAR(64)	PRIMARY KEY	Restriction record ID (e.g. RESTR-24H-1001).
customer_id	VARCHAR(64)	NOT NULL	Target customer ID.
incident_id	VARCHAR(64)	NULLABLE	Source fraud incident reference.
restriction_type	VARCHAR(64)	NOT NULL	TEMPORARY_RESTRICTION , TEMPORARY_PROTECTION , ADMIN_REVIEW .
status	VARCHAR(32)	DEFAULT 'ACTIVE'	Status: ACTIVE , RELEASED , EXPIRED .
started_at	TIMESTAMP	NOT NULL	Restriction enforcement start time.
expires_at	TIMESTAMP	NULLABLE	Expiration timestamp (e.g. +24 hours).

4. customer_security Table

Field	Type	Attributes	Description
customer_id	VARCHAR(64)	PRIMARY KEY	Customer reference ID.
security_status	VARCHAR(32)	DEFAULT 'NORMAL'	Priority status: NORMAL , STEP_UP_REQUIRED , PROTECTED , RESTRICTED , BANNED .
active_restriction_id	VARCHAR(64)	NULLABLE	Current active restriction record ID.
failed_login_count	INT	DEFAULT 0	Consecutive failed login counter.
step_up_required	BOOLEAN	DEFAULT FALSE	Indicates step-up OTP verification requirement.

5. verification_challenges Table

Field	Type	Attributes	Description
challenge_id	VARCHAR(64)	PRIMARY KEY	Verification challenge ID (e.g. CHAL-9921).
customer_id	VARCHAR(64)	NOT NULL	Customer ID.
destination_masked	VARCHAR(64)	NULLABLE	Masked phone reference (e.g. +91 3210).
otp_hash	VARCHAR(255)	NOT NULL	Encrypted hash of the 6-digit OTP code.
status	VARCHAR(32)	DEFAULT 'PENDING'	PENDING , VERIFIED , EXPIRED , FAILED .
attempts	INT	DEFAULT 0	Attempt counter (max 3).

7. Web Application Sitemap & User Interface Modules

7.1 Customer E-Commerce Experience Routes

Page Route	Purpose	Key Components & Features
/	E-Commerce Landing Page	Trust Bar, Promotional Banners, Category Carousel, Featured Products.
/login	Customer Authentication	Email/password login, automatic security check, redirect on restriction/OTP.
/register	Customer Sign-Up	New account registration with default NORMAL security status.
/products	Catalog Browser	Filtering by price, category, rating, brand, and sorting.
/product/:id	Product Details Page	Image gallery, variant selection, price discount badges, reviews list.
/cart	Shopping Cart	Item qty increment/decrement, subtotal calculation, coupon entry.
/checkout	Checkout & Payment	Shipping address selector, payment method selection, order placement.
/orders	Customer Order History	Historical orders list, invoice download trigger, item return request.
/security-verification	Step-Up OTP Verification	6-digit OTP input, resend challenge, demo code preview, attempt counter.
/account-restricted	Restricted Account Notice	24-hour restriction countdown timer, restriction reason display.
/account-banned	Permanent Ban Screen	Permanent ban notification with security ops contact reference.

7.2 FraudGuard Admin Security Ops Center (/admin/fraud/*)

Admin Subview	Purpose & Key Features
Dashboard	5 Live KPI Cards (Total Incidents, Admin Reviews Required, Banned Accounts, Currently Restricted, Step-Up Verification Pending), 7-Day Trend Chart with hover tooltips, Horizontal Workflow Bar Chart, Fraud Type Distribution.
Admin Reviews	Review queue for high-risk incidents (Score 85+). Actions: Extend Restriction (24h), Deactivate Account, Ban Account, Clear Review. Search bar included.
Incidents Registry	Complete paginated incident registry with severity filtering, decision action badges, risk score visualizer, CSV export.
Customers Registry	Comprehensive security profiles of registered customers with risk score history and timeline.
Banned Accounts	Registry of permanently banned customer accounts with Revoke Ban & Activate action button and search bar.
Restrictions	Active and historical account restrictions registry with Release Restriction action button.
Verification Registry	List of unique customer accounts requiring OTP verification with Simulate OTP and Open OTP UI actions.

8. API Specification & System Contracts

8.1 Public E-Commerce & Customer Endpoints

Method	Endpoint	Description	Auth Required
POST	/api/auth/register	Register new customer account.	Public
POST	/api/auth/login	Authenticate customer, perform security status check.	Public
GET	/api/products	Fetch paginated product catalog with filters.	Public
GET	/api/categories	List all e-commerce categories and subcategories.	Public
GET	/api/cart	Get customer cart items.	Session Token
POST	/api/checkout/place-order	Process checkout and create order.	Session Token
GET	/api/orders	Retrieve customer order history.	Session Token
GET	/api/invoices/:orderId	Download text/PDF invoice for confirmed order.	Session Token

8.2 FraudGuard & OTP Security Endpoints

Method	Endpoint	Description	Auth Required
GET	/api/fraud/overview	Fetch live KPI metrics and trend aggregation.	Admin Token
GET	/api/fraud/incidents	Fetch paginated fraud incidents from database.	Admin Token
GET	/api/fraud/reviews	Fetch pending admin review queue.	Admin Token
GET	/api/fraud/banned-accounts	Fetch permanently banned customer accounts.	Admin Token
GET	/api/fraud/restrictions	Fetch active/historical customer restrictions.	Admin Token
POST	/api/fraud/otp/request	Issue new OTP challenge for step-up verification.	Public / Admin
POST	/api/fraud/otp/verify	Verify 6-digit OTP code and update security status.	Public / Admin
POST	/api/fraud/admin/unban	Revoke permanent ban and activate customer account.	Admin Token
POST	/api/fraud/admin/release-restriction	Release active restriction and set customer status to ACTIVE .	Admin Token

9. Docker & Containerization Blueprint

9.1 Multi-Stage Production Docker file

```
# Multi-stage Dockerfile for RetailHub & FraudGuard Platform

# Stage 1: Build React Frontend
FROM node:20-alpine AS frontend-builder
WORKDIR /app/frontend
COPY frontend/package*.json ./
RUN npm ci
COPY frontend/ ./
RUN npm run build

# Stage 2: Build TypeScript Backend API
FROM node:20-alpine AS backend-builder
WORKDIR /app/backend
COPY backend/package*.json ./
RUN npm ci
COPY backend/ ./
RUN npm run build

# Stage 3: Production Container Runner
FROM node:20-alpine AS runner
WORKDIR /app

ENV NODE_ENV=production
ENV PORT=5000

# Copy production backend dependencies
COPY backend/package*.json ./backend/
RUN cd backend && npm ci --omit=dev

# Copy compiled backend dist
COPY --from=backend-builder /app/backend/dist ./backend/dist

# Copy compiled frontend static bundle for Express SPA serving
COPY --from=frontend-builder /app/frontend/dist ./frontend/dist

EXPOSE 5000

# Health check endpoint
HEALTHCHECK --interval=30s --timeout=10s --start-period=10s --retries=3 \
  CMD wget --no-verbose --tries=1 --spider http://localhost:5000/api/health || exit 1

CMD ["node", "backend/dist/index.js"]
```

9.2 Docker Compose Production Specification (docker-compose.yml)

```
version: '3.8'

services:
  app:
    image: naveen9200/website_app:latest
    container_name: nexday-platform
    restart: unless-stopped
    ports:
      - "5000:5000"
    env_file:
      - .env
    volumes:
      - nexday-event-logs:/app/backend/event_logs
    healthcheck:
      test: ["CMD-SHELL", "wget --no-verbose --tries=1 --spider http://localhost:5000/api/health || exit 1"]
      interval: 30s
      timeout: 10s
      retries: 3
      start_period: 10s

volumes:
  nexday-event-logs:
    driver: local
```

10. Operational Playbook & CLI Commands Summary

10.1 Local Development Execution

```
# Start backend server (TypeScript watch mode)
cd backend
npm run dev

# Start frontend application (Vite dev server)
cd frontend
npm run dev
```

10.2 Production Build Verification

```
# Compile and build frontend React bundle
cd frontend
npm run build

# Check backend TypeScript compilation
cd backend
npx tsc --noEmit
```

10.3 Docker Image Build & Push to Docker Hub

```
# Login to Docker Hub
docker login

# Build multi-stage Docker image
docker build -t naveen9200/website_app:latest -f Dockerfile .

# Push image to Docker Hub
docker push naveen9200/website_app:latest
```

10.4 Launching via Docker Compose

```
# Start pre-built platform container in background
docker compose up -d

# Check running container status
docker compose ps

# View live container logs
docker compose logs -f app
```

End of Technical Specification Book

RetailHub E-Commerce & FraudGuard Telemetry Engine Platform